

CNNs and Transfer Learning on CIFAR-10

Mohammed Yaghi Zaiter
Master 2 AI - Programming for AI (509)

Abstract—In this project we train convolutional neural networks (CNNs) on the CIFAR-10 dataset. First we build a baseline CNN from scratch, which reaches 74.9% test accuracy but overfits. Then we test the effect of the depth, kernel size, stride, padding, batch normalization, optimizer, learning rate, scheduler and data augmentation, changing only one thing at a time. The best combination reaches 82.4% test accuracy. Finally we use a ResNet-18 pre-trained on ImageNet, first frozen (82.7%) and then fine-tuned (95.6%). Fine-tuning converges much faster and still gives 88.9% when using only 10% of the training data.

Index Terms—CNN, CIFAR-10, data augmentation, batch normalization, transfer learning, fine-tuning

I. INTRODUCTION

The performance of a CNN depends on many choices: the number of layers, the kernel size, how the image is downsampled, the optimizer, the learning rate and the data augmentation. Another option is to not train from scratch and to reuse a network that was already trained on a big dataset like ImageNet (transfer learning).

The goal of this project is to measure the effect of these choices on CIFAR-10. The work is divided in 3 tasks:

- 1) a baseline CNN trained from scratch, with learning curves and a confusion matrix;
- 2) experiments on the architecture, the hyperparameters and the data augmentation;
- 3) transfer learning with a pre-trained ResNet-18 (frozen and fine-tuned) compared to our best CNN.

The code is written in PyTorch in Jupyter notebooks. All trained models are saved as checkpoints (.pth), so they can be loaded again and tested without training.

II. DATASET AND PIPELINE

CIFAR-10 has 60,000 color images of size 32×32 in 10 classes (plane, car, bird, cat, deer, dog, frog, horse, ship, truck). We split the 50,000 training images into 45,000 for training and 5,000 for validation (fixed seed). The 10,000 test images are only used at the end to evaluate the best model of each experiment.

The images are converted to tensors and normalized with the mean and standard deviation of each channel of CIFAR-10:

$$x_{norm} = \frac{x - mean}{std} \quad (1)$$

For ResNet-18 the images are resized to 128×128 and normalized with the ImageNet mean and std, because the pre-trained weights expect this kind of input.

We use 3 levels of data augmentation (only on the training images):

- **basic**: random crop (padding 4) and random horizontal flip;
- **full**: basic + random rotation/translation and color jitter;
- **MixUp**: full + mixing two images of the batch: $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$, with the loss mixed in the same way.

All models use the cross-entropy loss and a batch size of 128. For each epoch we save the train loss and accuracy, the validation loss and accuracy and the accuracy of each class. The model with the best validation accuracy is saved and then tested on the test set. The **generalization gap** is the train accuracy minus the validation accuracy at the best epoch. The **latency** is the time to predict one image. Training was done on Google Colab (T4 GPU), and the saved models were then tested again on a laptop CPU (the latency is measured on this CPU).

III. TASK 1: BASELINE CNN

The baseline has 3 convolution layers (3×3 kernels, padding 1), each followed by ReLU and a 2×2 max pooling, and 2 fully connected layers (Table I). The output size of a convolution is:

$$h_{out} = \frac{h_{in} + 2p - k}{s} + 1 \quad (2)$$

so with padding 1 the size stays the same, and each max pooling divides it by 2 ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$). The model has 620,362 parameters, and most of them (85%) are in the first fully connected layer. It is trained for 20 epochs with Adam (learning rate 0.001), without batch normalization, dropout or augmentation.

TABLE I
BASELINE CNN

Layer	Output shape
Input image	$3 \times 32 \times 32$
conv1 (32 filters) + ReLU + pool	$32 \times 16 \times 16$
conv2 (64 filters) + ReLU + pool	$64 \times 8 \times 8$
conv3 (128 filters) + ReLU + pool	$128 \times 4 \times 4$
Flatten	2048
fc1 + ReLU	256
fc2	10

Fig. 1 shows that the model overfits. The validation loss is minimum at epoch 6 and then goes up to 1.60, while the train loss goes down to 0.07. The best validation accuracy is 76.0% (epoch 12) but the train accuracy is already 95.1%, so the gap is 0.19. The test accuracy of the best model is 74.9%.

In the confusion matrix (Fig. 2) the most difficult classes are cat (57%) and dog (58%), and they are mostly confused with each other: 18% of the dogs are predicted as cats and 13% of the cats as dogs. Other common mistakes are truck

predicted as car (9%) and bird and deer confused with each other (8%). Car (86%) and ship (85%) are the easiest classes.

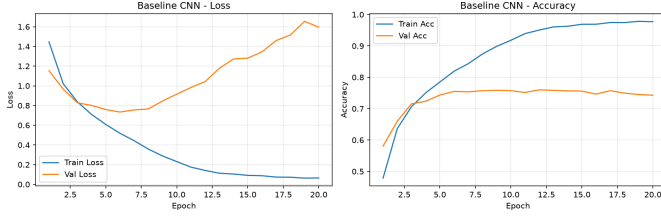


Fig. 1. Baseline: loss and accuracy curves

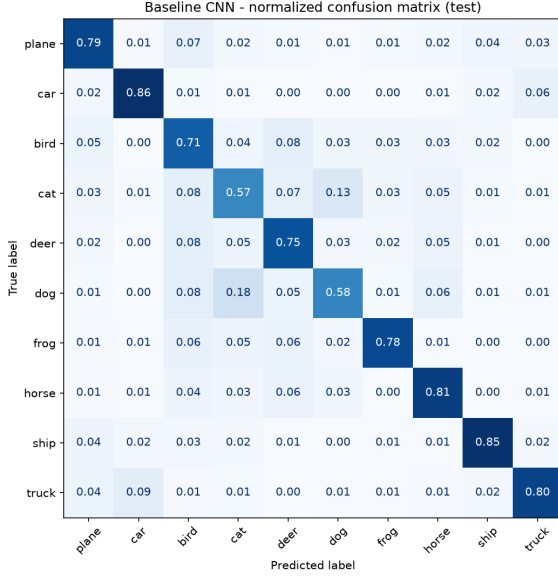


Fig. 2. Baseline: normalized confusion matrix (test set)

IV. TASK 2: ARCHITECTURE, HYPERPARAMETERS AND AUGMENTATION

Each experiment starts from the baseline and changes only one thing. The seed, the data split and the number of epochs (20) stay the same, so the difference in the results comes only from the changed setting. The baseline is trained again with the same code as the other experiments (75.3% test). SGD is used with learning rate 0.01 and momentum 0.9, and the learning rate tests are done with Adam. All results are in Table II.

A. Depth

With only 2 conv layers the test accuracy goes down to 72.3%. The number of parameters increases (1.07M) because the feature map before the first linear layer is bigger ($64 \times 8 \times 8$), and the receptive field is only 10 pixels, which is too small to see the whole object. With 5 conv layers the receptive field is 94 pixels, but the accuracy does not improve (75.5%) and the model overfits more. With only 1 linear layer the model has 5 times fewer parameters and a smaller gap (0.095), and loses only 1.2 points. With 4 linear layers there are more parameters and more overfitting. So most parameters of the baseline are in

the fully connected part, and they are mostly used to memorize the training data.

B. Kernel Size

Bigger kernels give worse results: 73.3% with 5×5 and 69.4% with 7×7 , with more parameters and a slower model. On small 32×32 images, several 3×3 layers already cover the object with fewer parameters, and big kernels lose small details.

C. Stride and Padding

Using a convolution with stride 2 instead of max pooling gives the worst architecture (67.9%). The receptive field is smaller (15 pixels) and the stride skips half of the positions, while max pooling keeps the strongest value of each 2×2 region. It is the fastest model (0.16 ms per image). Without padding the feature maps become very small (2×2), the model has only 227K parameters and less overfitting, but it loses 2.2 points because the information at the borders is lost.

D. Batch Normalization

Batch normalization is the best change on the architecture: 77.4% (+2.1 points) with only 448 more parameters. It normalizes the outputs of each layer, which makes the training more stable. The best epoch is also later (11 instead of 8).

E. Optimizer and Learning Rate

SGD with momentum gives 75.9%, but it is still improving at epoch 20 and it almost memorizes the training set (98.7% train accuracy). AdamW gives the same result as Adam (75.5%) with a slightly smaller gap.

The learning rate has the biggest effect. With Adam and $lr = 0.01$ the training does not work at all: the loss stays at 2.303, which is $\ln(10)$, the loss of a random guess between 10 classes, and the model predicts the same class for all images (10% accuracy). The updates are too big and the model cannot recover. With $lr = 0.0001$ the training is too slow: 69.4% after 20 epochs, and it is still improving.

F. Schedulers

Decreasing the learning rate during training improves the result by about 2 points (77.3% with StepLR, 76.9% with cosine). A big learning rate at the start learns fast, and a small learning rate at the end helps to converge.

G. Data Augmentation

Augmentation gives the biggest improvement. With crop and flip the test accuracy is 80.5% (+5.2 points) and the gap is almost 0 (Fig. 3). The validation loss keeps decreasing instead of going up like in the baseline. The full augmentation gives 78.0%: the train accuracy is lower than the validation accuracy, so 20 epochs are not enough for this harder task. It is also 2 times slower (32 s per epoch instead of 14 s). MixUp gives 77.9%; its train accuracy is computed on mixed images, so its gap cannot be compared.

TABLE II
TASK 2 RESULTS (RF = RECEPTIVE FIELD IN PIXELS, LATENCY ON CPU, GAP = TRAIN ACC - VAL ACC AT THE BEST EPOCH)

Group	Experiment	Params	RF	Latency (ms)	Best val acc	Gap	Test acc
Baseline	3 conv, 3×3 , max pool, Adam 0.001	620,362	22	0.35	0.768	0.134	0.753
Depth	2 conv layers	1,070,794	10	0.30	0.736	0.187	0.723
	5 conv layers	1,046,858	94	0.53	0.764	0.202	0.755
	1 linear layer	113,738	22	0.32	0.758	0.095	0.741
	4 linear layers	1,307,850	22	0.40	0.758	0.194	0.746
Kernel	5×5	785,738	36	0.79	0.752	0.168	0.733
	7×7	1,033,802	50	0.94	0.711	0.148	0.694
Stride/padding	stride 2 instead of max pool	620,362	15	0.16	0.689	0.178	0.679
	no padding	227,146	22	0.24	0.743	0.075	0.731
BatchNorm	BatchNorm after each conv	620,810	22	0.49	0.785	0.148	0.774
Optimizer	SGD + momentum (lr 0.01)	620,362	22	0.39	0.767	0.220	0.759
	AdamW (lr 0.001)	620,362	22	0.46	0.766	0.128	0.755
Learning rate	Adam lr 0.01	620,362	22	0.45	0.106	-0.008	0.100
	Adam lr 0.0001	620,362	22	0.46	0.698	0.025	0.694
Scheduler	StepLR	620,362	22	0.51	0.782	0.146	0.773
	Cosine	620,362	22	0.46	0.775	0.225	0.769
Augmentation	basic (crop + flip)	620,362	22	0.47	0.817	0.007	0.805
	full	620,362	22	0.47	0.797	-0.044	0.780
	full + MixUp	620,362	22	0.49	0.787	-0.375	0.779
Final	best combination (30 epochs)	1,423,114	22	1.24	0.838	-0.050	0.824

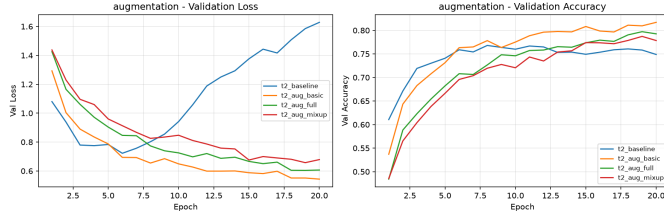


Fig. 3. Validation loss and accuracy with data augmentation

H. Best Combination

The final model uses wider layers (64, 128, 256 filters), batch normalization, dropout 0.3, AdamW with a cosine scheduler and full augmentation, for 30 epochs. It reaches 82.4% on the test set (+7.1 points compared to the baseline) and does not overfit (Fig. 4). The best epoch is 29 out of 30, so training longer could still help.

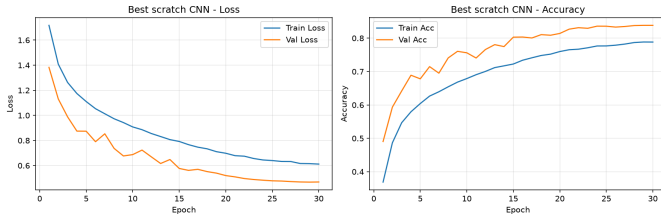


Fig. 4. Best CNN from scratch: loss and accuracy curves

V. TASK 3: TRANSFER LEARNING

We use a ResNet-18 pre-trained on ImageNet and replace its last layer by a new one for 10 classes (dropout 0.5 + linear layer). We compare 3 methods with our best CNN from Task 2:

- **frozen:** all layers of ResNet have `requires_grad = False`, only the new last layer is trained (5,130 parameters, lr = 0.001);
- **fine-tune layer4:** the last block of ResNet and the new layer are trained (8.4M parameters, lr = 0.0001);
- **fine-tune all:** all the network is trained (11.2M parameters, lr = 0.0001).

For fine-tuning we use a 10 times smaller learning rate, so the pre-trained weights are only adjusted and not destroyed. All models are trained for 10 epochs with AdamW, a cosine scheduler and basic augmentation, once with 100% and once with 10% of the training data (4,500 images).

TABLE III
TASK 3 RESULTS (TEST ACCURACY)

Method	Trained params	10% data	100% data	s/epoch
CNN from scratch	1.42M	0.570	0.791	18
Frozen	5.1K	0.786	0.827	46
Fine-tune layer4	8.40M	0.867	0.931	48
Fine-tune all	11.18M	0.889	0.956	58

A. Results

With 100% of the data (Table III), the frozen ResNet (82.7%) is already as good as our best CNN trained for 30 epochs (82.4%), while training only 5,130 parameters. Fine-tuning is much better: 93.1% for layer4 and 95.6% for the whole network. The ImageNet features (edges, textures, shapes) are useful for CIFAR-10, and adapting the last layers to our images gives more than 10 extra points. With fine-tuning, cat and dog are much better recognized (91% and 93% on validation, instead of 54% and 71% for the CNN from scratch).

B. Convergence Speed

The pre-trained models are very fast to train (Fig. 5). After only 1 epoch, the frozen model is at 80% and the fine-tuned

model at 93% validation accuracy, while the CNN from scratch is at 56% and never reaches 80% in 10 epochs.

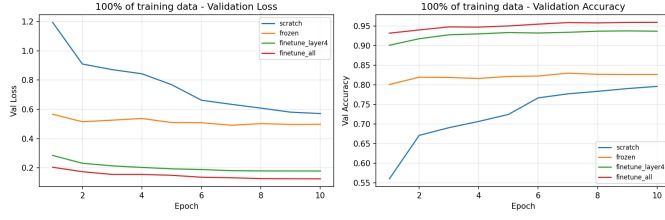


Fig. 5. Validation loss and accuracy of the 4 methods (100% data)

C. Data Efficiency

With only 10% of the data, the CNN from scratch loses 22.2 points (57.0%), while the frozen and fine-tuned models lose only 4 to 7 points (Fig. 6). With 4,500 images, full fine-tuning (88.9%) is still better than our best CNN trained on all the data. Transfer learning is very useful when there is not a lot of data.

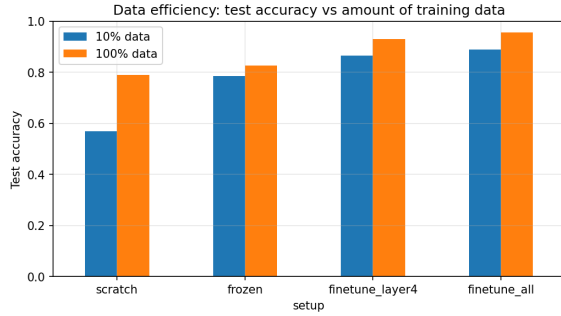


Fig. 6. Test accuracy with 10% and 100% of the training data



Fig. 7. Fine-tuned ResNet-18: normalized confusion matrix (test set)

D. Cost

ResNet-18 is bigger (11.2M parameters) and uses 128×128 images, so it is slower: 46 to 58 s per epoch instead of 18 s for the CNN from scratch. The frozen model is not much faster to train, because the images still have to go through all the layers.

VI. DISCUSSION

The baseline has enough parameters to memorize the training set, so the changes that help the most are the ones that reduce overfitting or make the training more stable: data augmentation (+5.2 points), batch normalization (+2.1) and the learning rate scheduler (+2). Adding parameters (bigger kernels, more linear layers) or losing information (stride instead of pooling, no padding) makes the results worse. The learning rate must be chosen carefully: 0.01 was too big for Adam and 0.0001 too small. Transfer learning is much better in accuracy, speed of convergence and data efficiency, but the model is bigger and slower.

Limitations: each experiment was trained only once (one seed), so small differences (less than 1 point) are not very meaningful. The number of epochs was fixed, which is not good for the slow methods (SGD, full augmentation, MixUp) that were still improving. ResNet-18 was used with 128×128 images instead of 224×224 to save time.

VII. CONCLUSION

We built a PyTorch pipeline for CIFAR-10 and tested many choices for CNNs. The baseline overfits (74.9%); batch normalization, learning rate schedulers and data augmentation help the most, and together they give 82.4% with a CNN from scratch. Transfer learning with ResNet-18 gives 95.6% after fine-tuning, learns in 1 epoch and still gives 88.9% with only 10% of the data. As future work, we could train longer, try CutMix, use several seeds and fine-tune ResNet with 224×224 images.